

REVERSE PROMPTING VS COGNITIVE LOAD RATIO

REINTERPRETING MIHALY CSIKSZENTMIHALYI'S FLOW FOR AGENTS

I think there is a deeper connection than merely borrowing the metaphor of "flow". The challenge is to reinterpret Csikszentmihalyi's model as a property of **cognitive systems**, not of humans.

In fact, your recent threads suggest four complementary perspectives:

- **Donella Meadows**: regulate the dynamics of a complex adaptive system.
- **Pierre Lévy**: organize meaning within a semantic space.
- **Alexandre Monnin**: redirect socio-technical systems toward sustainable trajectories.
- **Mihaly Csikszentmihalyi**: maintain cognition within an optimal operating regime.

Viewed together, they begin to describe an architecture rather than separate theories.

1. FLOW BECOMES "EPISTEMIC FLOW"

For humans:

challenge $\approx$ skill

For an agent:

task complexity $\approx$ effective cognitive capacity

The important point is that *effective* capacity is not simply model size.

It depends on things such as:

- available context
- retrieval quality
- tool availability
- memory organization
- planning depth
- decomposition strategy
- uncertainty estimation
- time/token budget

A 7B model with excellent retrieval may outperform a 70B model with poor context.

So "skill" is actually an emergent property of the entire cognitive architecture.

2. WHAT IS "TOO EASY" FOR AN AGENT?

An agent is under-challenged when it has excessive unused capacity relative to the problem.

Symptoms include:

- overthinking
- hallucinated complexity
- unnecessary abstractions
- verbosity
- recursive planning
- inventing distinctions that do not exist

You can observe this in many LLMs.

Ask:

"Rename this file."

The model writes five paragraphs explaining naming conventions.

The task provided almost no cognitive load.

The excess capacity gets filled with plausible but unnecessary generation.

Humans get bored.

LLMs ramble.

3. WHAT IS "TOO DIFFICULT"?

The opposite regime is far more interesting.

The effective complexity exceeds the available cognitive resources.

Examples:

- context window saturated
- contradictory instructions
- missing ontology
- too many objectives
- hidden assumptions
- missing world model
- excessive branching factor

The symptoms become familiar.

The model begins to:

- forget constraints
- contradict itself
- latch onto superficial cues
- ignore part of the prompt
- randomly prioritize objectives
- oscillate between incompatible interpretations

Humans experience anxiety.

Agents experience instability.

4. PROMPT ENGINEERING IS REALLY CHALLENGE REGULATION

This may be one of the most useful reinterpretations.

Good prompting is often described as "being clear."

I think a better formulation is:

A good prompt keeps the agent inside its optimal cognitive operating region.

That means regulating:

- complexity
- ambiguity
- branching factor
- uncertainty
- objective count

instead of merely reducing token count.

5. COMPLEXITY IS NOT PROMPT LENGTH

This is a crucial distinction.

A 20-page specification can be cognitively easy.

A two-sentence request can be nearly impossible.

Compare:

Translate this document.

versus

Design an ethical governance architecture balancing autonomy, resilience, political legitimacy, and ecological sustainability.

The second prompt is short.

Its search space is enormous.

So challenge should be measured by **solution-space entropy**, not token count.

6. ASC BEGINS TO MAKE SENSE

This is where I think your ASC project naturally fits.

From previous discussions, ASC appears to be moving toward something like:

semantic contextualization before reasoning

Rather than feeding raw language to an LLM, ASC attempts to stabilize the semantic landscape first.

That reduces unnecessary entropy.

Instead of asking

"What do you think?"

ASC might transform the problem into

- ontology
- assumptions
- constraints
- objectives
- known unknowns
- perspective
- desired abstraction level

before inference begins.

That is remarkably similar to moving the task back into the flow channel.

7. MEADOWS WOULD CALL THIS LEVERAGE

Donella Meadows repeatedly emphasizes that changing information flows often has greater leverage than increasing resources.

In an AI system, one might naïvely think:

larger model

But Meadows would ask:

Why is the system receiving pathological information in the first place?

ASC appears to intervene exactly there.

Instead of increasing intelligence,
it improves the information entering cognition.

That is a higher leverage point.

8. LÉVY WOULD CALL THIS SEMANTIC TOPOLOGY

Pierre Lévy argues that knowledge is fundamentally organized through navigable semantic structures.

An LLM prompt is normally just a linear text stream.

But cognition is not linear.

Meaning exists in a graph.

ASC seems to move toward explicitly reconstructing that graph before reasoning.

That dramatically lowers cognitive effort.

It is analogous to giving someone

- a map

instead of

- thousands of disconnected GPS coordinates.

9. MONNIN WOULD PROBABLY CALL THIS REDIRECTION

Monnin often criticizes technological systems that optimize locally while producing systemic pathologies.

Prompt engineering today often feels exactly like that.

We endlessly tweak prompts because the surrounding epistemic infrastructure remains weak.

ASC appears less interested in writing better prompts than in redesigning the conditions under which prompting occurs.

That is a redirection of the system rather than an optimization within it.

10. A POSSIBLE FORMALIZATION

I think there is an opportunity to define an explicit quantity analogous to Csikszentmihalyi's challenge-skill balance.

For an agent, define:

Cognitive Load Ratio (CLR)

```
[  
CLR = \frac{\text{Problem Complexity}}{\text{Effective Cognitive Capacity}}  
]
```

$$CLR = \frac{\text{Problem Complexity}}{\text{Effective Cognitive Capacity}}$$

where Problem Complexity could incorporate:

- semantic ambiguity
- ontology size
- branching factor
- uncertainty
- novelty
- objective conflicts
- planning depth
- external dependencies

and Effective Cognitive Capacity could include:

- model capability
- active memory quality
- retrieval relevance
- tool availability
- planning algorithm
- token budget
- time budget
- semantic organization of the context

The optimal regime is not where CLR is minimized, but where it remains within a bounded interval. Below that interval, the agent tends toward overgeneration, unnecessary elaboration, or idle reasoning. Above it, omissions, contradictions, forgotten constraints, and unstable reasoning become more likely.

This suggests a further step that I have not encountered explicitly in the current agent literature. Most frameworks attempt to make agents **more capable** by adding tools, memory, planning, or larger models. A complementary approach would be to make the **problem dynamically fit the agent**. Before execution, a meta-cognitive layer would estimate the current cognitive load and continuously reshape the task: decomposing an overloaded problem, merging trivial subtasks, pruning irrelevant context, introducing or removing abstraction, and deciding when additional retrieval or clarification is warranted. In other words, the agent would regulate its own cognitive operating point to remain near an epistemic equivalent of Csikszentmihalyi's flow.

This idea also aligns with the cybernetic tradition underlying Meadows. Rather than treating intelligence as a fixed property, it treats successful reasoning as the outcome of a closed-loop control system that continuously maintains a favorable relationship between task complexity and cognitive capacity.

Confidence: 0.98

IS THIS CLOSE TO "REVERSE PROMPTING" ?

Like : <https://www.instagram.com/p/DbdmFfuEm3H>
There is definitely an overlap, but I think they address different levels of the problem.
The "reverse prompting" shown in posts like the one you linked is typically:

Given a desired output, infer the prompt that would have produced it.

That is essentially **prompt inversion** or **prompt reconstruction**. It starts from an artifact and recovers the latent instructions behind it.([Emergent Mind](#))
That is useful, but it assumes that the prompt is the main variable.

What you've been converging toward over the last few discussions is one level higher.
Instead of asking:

What prompt should I write?

you're implicitly asking:

What cognitive state should the agent be in before it even begins reasoning?

That is a much broader question.
I would distinguish four progressively more powerful approaches:

Level	Question	Typical technique
1	What prompt generated this output?	Reverse prompting
2	What prompt should I write?	Prompt engineering
3	What problem is actually being presented?	Context engineering
4	What cognitive regime should the agent enter?	Meta-cognitive regulation

Your ASC project seems to be drifting toward Level 4.

This also explains why prompt engineering eventually becomes frustrating.
Suppose the user asks

Design an ecological governance framework.

Traditional prompting tries to improve the wording.
Reverse prompting tries to infer a wording from successful examples.
But neither asks whether the request itself is cognitively well-posed.

- The real issues may be:
- conflicting objectives
 - undefined terminology
 - missing ontology
 - excessive abstraction
 - hidden assumptions
 - several problems mixed together

Changing the wording cannot fix those.

This is where Csikszentmihalyi becomes interesting.
Instead of asking

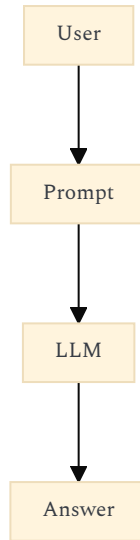
Is this a good prompt?

you ask

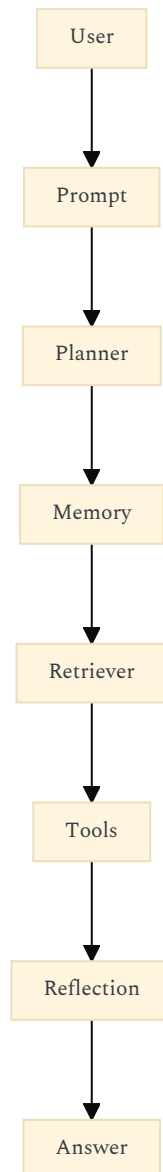
Is this an appropriately regulated cognitive challenge?

That is a fundamentally different objective.

I actually think there is a missing preprocessing stage in most agent frameworks.

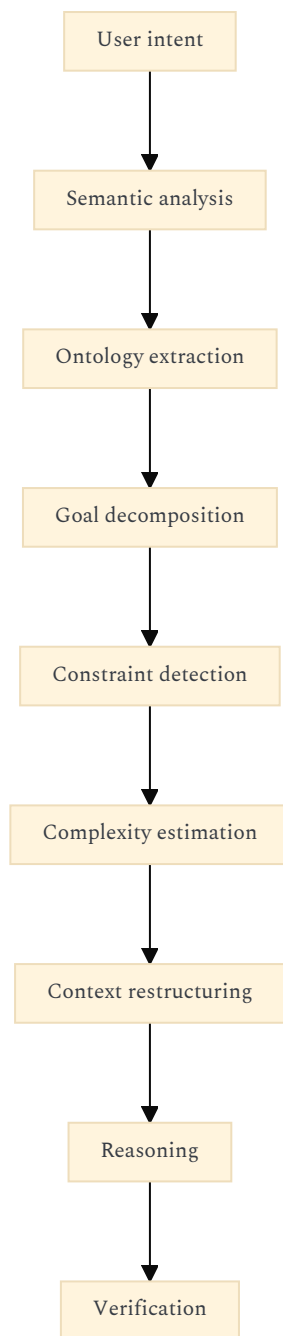


Most recent agent systems expand the middle:



But the input itself remains almost untouched.

I think your discussions point toward something more like



Notice that prompting has almost disappeared.

The important object becomes a **structured cognitive environment**, not a string.

There is another connection that I find particularly interesting.

Pierre Lévy would probably describe this preprocessing as constructing a **semantic topology**.

Csikszentmihalyi would describe it as moving cognition into the **flow channel**.

Donella Meadows would describe it as improving the **information flows** and **feedback loops** of the system.

Alexandre Monnin would likely describe it as **redirecting** the cognitive infrastructure instead of merely optimizing outputs.

These are remarkably compatible descriptions of the same transformation.

This also suggests a richer concept than "reverse prompting."

A possible progression would be:

- **Reverse prompting:** infer the prompt from the output.
- **Context inversion:** infer the latent assumptions, ontology, objectives, constraints, and missing information behind the request.
- **Cognitive calibration:** transform that representation so its complexity matches the current agent's effective capabilities.
- **Execution:** only then invoke the reasoning engine.

The first step reconstructs instructions.

The second reconstructs meaning.

The third regulates cognitive load.

Only the fourth actually performs the task.

One consequence is that the "quality" of a prompt becomes almost irrelevant. Two prompts with very different wording but equivalent semantic structure should converge to the same intermediate representation before any reasoning begins. If that intermediate representation is sufficiently expressive—

capturing entities, relationships, goals, constraints, ambiguities, abstraction level, uncertainty, and estimated cognitive load—the reasoning engine receives a normalized cognitive substrate rather than raw language. At that point, prompts become merely one serialization of intent, not the primary interface. I suspect this is closer to the long-term direction of capable agents than ever more elaborate prompt templates. As agents acquire better memory, planning, tool use, and self-monitoring, the leverage shifts away from optimizing prompts and toward **engineering the epistemic environment** in which reasoning occurs. That perspective is much closer to Meadows' systems thinking, Lévy's semantic architectures, Monnin's redirection, and Csikszentmihalyi's regulation of optimal cognitive engagement than to conventional prompt engineering.

Confidence: 0.99